

**Université Cadi Ayyad
École Normale Supérieure
Département d'Informatique**

Rapport de projet de fin de module

*Projet : Mise en place d'une application web et mobile pour le suivi du
transport scolaire*



Réalisé par :

Rhendour Zakaria

Encadré par :

Pr. M. LACHGAR

Année Universitaire : 2025–2026

Déclaration sur l'honneur

Je soussigné, **Rhendour Zakaria**, étudiant à l'École Normale Supérieure, Université Cadi Ayyad, filière CLE-INFO, déclare que ce rapport, y compris les textes, figures, tableaux, extraits de code et illustrations, est le résultat de mon travail personnel. Toute source externe utilisée dans la préparation de ce document est citée et référencée explicitement. Je reconnais que toute violation de cette déclaration peut être considérée comme un plagiat académique et peut entraîner des mesures disciplinaires.

J'autorise la mise à disposition d'une copie académique de ce rapport à des fins pédagogiques, au bénéfice des futurs étudiants et chercheurs, et j'accepte qu'il puisse être consulté dans l'intérêt général de l'enseignement supérieur et de la recherche.

Rhendour Zakaria

14 mai 2026

Remerciements

Je tiens d'abord à remercier ma famille pour sa patience, ses encouragements et sa confiance tout au long de ce projet. Son soutien m'a aidé à avancer pendant les longues étapes de développement, de test, de correction et de documentation.

Je remercie également mes amis et mes camarades pour leur aide, leurs remarques, leurs discussions et leur motivation. Leur présence a rendu le travail plus léger et m'a permis d'améliorer à la fois la partie technique de SafeBus et la présentation finale du rapport.

Enfin, je remercie **Pr. M. LACHGAR** pour son encadrement et ses remarques utiles, ainsi que la filière CLE-INFO de l'École Normale Supérieure, Université Cadi Ayyad, pour le cadre académique dans lequel ce projet a été réalisé.

Résumé

SafeBus est une application web et mobile conçue pour faciliter le suivi du transport scolaire. Le problème de départ est concret : dans plusieurs établissements, les informations concernant le départ, le retard, l'arrivée ou la position d'un bus sont encore communiquées manuellement. Cela peut créer de l'incertitude chez les parents et augmenter la charge de communication pour l'administration.

Le projet propose un prototype fonctionnel dans lequel l'administrateur gère les données de transport depuis un tableau de bord web, tandis que les chauffeurs et les parents utilisent une application Android adaptée à leurs rôles. La solution repose sur un backend Node.js et Express connecté à une base de données MySQL. Ce backend gère l'authentification par JSON Web Token, le contrôle d'accès par rôle, la validation des données, les opérations CRUD, les actions du chauffeur, les mises à jour GPS, la création des notifications et les événements en temps réel avec Socket.IO.

Le tableau de bord web est développé avec React et Vite. Il permet à l'administrateur de gérer les élèves, les bus, les trajets, les comptes chauffeurs, les comptes parents et les notifications. L'application mobile est développée nativement en Kotlin et utilise Retrofit, ViewModel, LiveData, RecyclerView et SharedPreferences pour assurer les parcours parent et chauffeur.

Le projet reste un prototype académique, mais il suit une structure full-stack complète : séparation du code source, données de démonstration, endpoints API, mises à jour en temps réel, exécution avec Docker Compose et documentation. Le résultat montré qu'une plateforme numérique simple mais cohérente peut réduire les problèmes de communication dans le transport scolaire et servir de base à des améliorations futures comme les notifications push, la gestion avancée des utilisateurs, l'historique des trajets, les tests automatisés et le déploiement cloud.

Mots-clés : transport scolaire, React, Android, Node.js, notifications en temps réel

Code source : <https://github.com/00zkr/safebus>

Synthese

Ce rapport présente SafeBus, un système de suivi du transport scolaire développé comme projet web et mobile complet. L'application est construite autour de trois profils qui n'ont pas besoin de la même interface : l'administrateur utilise un tableau de bord dans le navigateur, le chauffeur utilise un écran Android pendant le trajet, et le parent consulte l'état du bus de son enfant ainsi que l'historique des notifications depuis la même application mobile.

Au centre du système se trouve une API Express connectée à MySQL. Ce backend stocke les principales données de transport, vérifie les rôles des utilisateurs, valide les requêtes, génère les notifications et envoie des événements Socket.IO lorsqu'une information importante sur un bus change. Le tableau de bord React consomme cette API pour les tâches d'administration, tandis que l'application Android Kotlin l'utilise pour les interactions des chauffeurs et des parents.

L'objectif n'est pas de construire une solution industrielle de gestion de flotte, mais de livrer un prototype académique réaliste, suffisamment complet pour montrer l'analyse, l'architecture, l'implémentation, l'intégration et la validation. Le résultat est un système exécutable avec Docker Compose, des comptes de démonstration, des données initiales, des fonctionnalités par rôle et une base claire pour de futures extensions.

Liste des Abreviations

API	Interface de Programmation d'Application
CRUD	Create, Read, Update, Delete
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
JWT	JSON Web Token
MVC	Model View Controller
REST	Représentational State Transfer
SQL	Structured Query Language
UI	Interface Utilisateur
UML	Unified Modeling Language

Table des figures

4.1	Architecture globale de SafeBus	10
4.2	Diagramme de cas d'utilisation administrateur	11
4.3	Diagramme de cas d'utilisation chauffeur	11
4.4	Diagramme de cas d'utilisation parent	14
4.5	Diagramme de classes de la structure backend	15
4.6	Diagramme de classes du modèle de domaine SafeBus	15
4.7	Diagramme de sequence de l'authentification	16
4.8	Diagramme de sequence de la gestion des données administrateur	16
4.9	Diagramme de sequence du démarrage de trajet chauffeur	17
4.10	Diagramme de sequence de la mise à jour GPS chauffeur	17
4.11	Diagramme de sequence de consultation des notifications parent	18
5.1	écran de connexion administrateur SafeBus	21
5.2	Dashboard administrateur avec les indicateurs de transport	22
5.3	écran de connexion Android	23
5.4	écran chauffeur capture depuis l'émulateur	24
5.5	écran parent capture depuis l'émulateur	25

Liste des tableaux

4.1	Rôles et permissions	12
4.2	Principales entités de données	12
5.1	Utilisateurs de démonstration	20
5.2	Principaux endpoints API	20

Table des matières

Remerciements	ii
Résumé	iii
Synthese	iv
Liste des Abreviations	v
Table des figures	vi
Liste des tableaux	vii
Introduction Générale	1
1 Contexte du Projet et Besoins	3
1.1 Introduction	3
1.2 Contexte du Projet	3
1.3 Utilisateurs Cibles	3
1.3.1 Administrateur	3
1.3.2 Chauffeur	3
1.3.3 Parent	4
1.4 Besoins Fonctionnels	4
1.5 Besoins Non Fonctionnels	4
1.6 Contraintes	4
1.7 Conclusion	5
2 Notions Techniques Preliminaires	6
2.1 Introduction	6
2.2 API REST	6
2.3 Authentification par JSON Web Token	6
2.4 Communication en Temps Réel	6
2.5 Base de Données Relationnelle	7
2.6 Tableau de Bord Web	7
2.7 Application Android Native	7
2.8 Docker Compose	7
2.9 Conclusion	7
3 Travaux Connexes et Choix Technologiques	8
3.1 Introduction	8
3.2 Systèmes de Suivi de Transport	8
3.3 Choix des Technologies Backend	8
3.4 Choix des Technologies Web	9

3.5	Choix des Technologies Mobiles	9
3.6	Choix de Déploiement	9
3.7	Conclusion	9
4	Analyse et Conception	10
4.1	Introduction	10
4.2	Architecture Globale	10
4.3	Rôles et Permissions	10
4.4	Conception en Couches du Backend	10
4.5	Modèle de Données	12
4.6	Workflows Principaux	12
4.6.1	Workflow d'Authentification	12
4.6.2	Workflow de Gestion Administrateur	13
4.6.3	Workflow de Mise à Jour du Chauffeur	13
4.6.4	Workflow de Position GPS	13
4.6.5	Workflow de Notification Parent	13
4.7	Conception de la Sécurité	13
4.8	Conclusion	13
5	Réalisation	19
5.1	Introduction	19
5.2	Réalisation du Backend	19
5.2.1	Structure Backend	19
5.2.2	Authentification	19
5.2.3	Principaux Endpoints API	19
5.3	Réalisation du Temps Réel	20
5.4	Réalisation du Tableau de Bord Web	20
5.4.1	Structure Web	20
5.4.2	Fonctionnalités du Dashboard	21
5.5	Réalisation de l'Application Android	21
5.5.1	Structure Mobile	21
5.5.2	Fonctionnalités Parent	22
5.5.3	Fonctionnalités Chauffeur	22
5.6	Réalisation de la Base de Données	22
5.7	Réalisation du Déploiement	23
5.8	Tests et Validation	23
5.9	Conclusion	24
6	Conclusion et Perspectives	26
6.1	Conclusion Générale	26
6.2	Resultats du Projet	26
6.3	Limites	26
6.4	Perspectives	27
6.5	Remarques Finales	27
7	Contribution Personnelle et Acquis	28
7.1	Introduction	28
7.2	Contribution Personnelle	28
7.3	Compétences Techniques Developpees	28
7.4	Difficultes Rencontrees	29
7.5	Lecons Apprises	29
7.6	Conclusion	29

Bibliographie	30
Appendices	31
A Reference API	31
A.1 URLs de Base	31
A.2 Exemple de Connexion	31
A.3 En-tête d'Autorisation	31
A.4 Exemple de Position Chauffeur	31
A.5 Commande Docker	31
B Dépôt du Projet et Notes de Livraison	32
B.1 Dépôt GitHub	32
B.2 Structure du Dépôt	32
B.3 Scénario de Demonstration Recommande	32
B.4 écrans Inclus dans le Rapport	32
C Diagrammes UML	34
C.1 Diagramme de Cas d'Utilisation	34
C.2 Diagrammes de Sequence	34
C.3 Diagrammes de Classes	34
C.4 Recommandation d'Integration	35

Introduction Générale

Le transport scolaire est un service quotidien où un simple retard d'information peut rapidement devenir source d'inquiétude. Un parent peut attendre sans savoir si le bus est déjà parti, un chauffeur peut devoir signaler un retard tout en restant concentré sur son trajet, et l'administration peut recevoir plusieurs appels parce que l'information n'est pas centralisée. Ces situations sont fréquentes lorsque le suivi du transport repose principalement sur des appels téléphoniques ou des messages informels.

SafeBus a été développé comme une réponse pratique à cette situation. Le projet ne cherche pas à couvrir toutes les fonctionnalités d'une solution professionnelle de gestion de flotte. Il se concentre plutôt sur les besoins essentiels d'un contexte scolaire : gestion des bus, des élèves et des trajets, mise à jour de l'état du bus par le chauffeur, partage de position GPS et notifications destinées aux parents. Le système final combine une API backend, un tableau de bord web, une application Android native, une base de données relationnelle et une communication en temps réel.

Problématique

La problématique principale traitée par ce projet est l'absence d'une vision partagée et actualisée du processus de transport scolaire. Sans outil dédié, chaque acteur ne dispose que d'une partie de l'information. L'administrateur connaît les bus et les élèves inscrits, le chauffeur connaît la situation sur la route, et le parent ne reçoit une information que si elle lui est transmise manuellement.

SafeBus centralise ces informations et les rend accessibles à travers des interfaces adaptées aux rôles. L'administrateur supervise les données depuis le tableau de bord. Le chauffeur envoie les mises à jour de progression et de position depuis l'application mobile. Le parent consulte les informations utiles concernant l'enfant et le bus qui lui sont associés. Le projet constitue ainsi un prototype académique réaliste d'une plateforme de suivi du transport scolaire.

Objectifs du Projet

L'objectif principal est de concevoir et implémenter un prototype full-stack complet pour le suivi du transport scolaire. Cet objectif se décline en plusieurs objectifs spécifiques :

- Développer une API backend qui gère l'authentification, l'accès aux données, la validation et les règles métier.
- Créer un tableau de bord web destiné au rôle administrateur.
- Créer une application Android native avec des parcours séparés pour les parents et les chauffeurs.
- Stocker les données de transport dans une base de données MySQL.
- Générer des notifications lorsque des actions chauffeur sont effectuées.
- Émettre des événements en temps réel pour permettre au tableau de bord de réagir aux changements.

- Fournir un environnement d'exécution local reproductible avec Docker Compose.
- Documenter le projet pour l'évaluation, la démonstration et la maintenance future.

Méthodologie

Le projet a été réalisé de manière incrémentale. La première étape a consisté à identifier les utilisateurs et les principaux parcours fonctionnels. Ensuite, le schéma de base de données et les couches backend ont été préparés. L'API a été implémentée avec l'authentification, la validation, les règles métier, la génération des notifications et les événements en temps réel. Le tableau de bord React a ensuite été connecté au backend, puis l'application Android a été développée pour les usages parent et chauffeur. La dernière étape a porté sur l'exécution avec Docker Compose, les données de démonstration et la documentation.

Organisation du Rapport

Ce rapport est organisé comme suit. Le chapitre 1 présente l'introduction générale. Le chapitre 2 détaille le contexte du projet, les besoins et les contraintes. Le chapitre 3 introduit les concepts techniques principaux. Le chapitre 4 explique les choix technologiques. Le chapitre 5 présente l'analyse et la conception. Le chapitre 6 décrit l'implémentation. Le chapitre 7 conclut le travail et propose des améliorations futures. Le chapitre 8 résume la contribution personnelle et les acquis du projet.

Chapitre 1

Contexte du Projet et Besoins

1.1 Introduction

Ce chapitre définit l'environnement dans lequel SafeBus a été développé. Il présente les utilisateurs, le comportement attendu du système et les contraintes qui ont influencé le prototype final.

1.2 Contexte du Projet

SafeBus est conçu pour une École ou un centre de formation qui organise le transport des élèves avec des bus et des trajets assignés. Dans le scénario du projet, l'établissement a besoin d'un moyen simple pour organiser les informations de transport et les rendre visibles aux personnes concernées. Le processus quotidien implique trois acteurs principaux :

- L'administrateur, qui organise les élèves, les trajets, les bus et les comptes.
- Le chauffeur, qui effectue le trajet et signale sa progression.
- Le parent, qui a besoin d'informations sur le bus associé à son enfant.

Pour cette version académique, le contexte utilisé est Toubkal IT Center. Le prototype garde un domaine volontairement limité : les élèves sont associés à des bus et à des trajets, les chauffeurs signalent l'avancement du trajet, et les parents consultent l'état ainsi que les notifications liées à leur enfant.

1.3 Utilisateurs Cibles

1.3.1 Administrateur

L'administrateur utilise le tableau de bord web. Il prépare et supervise les données de transport : élèves, bus, trajets, comptes chauffeurs, comptes parents et notifications. Le tableau de bord permet aussi d'observer les mises à jour en temps réel pendant la démonstration.

1.3.2 Chauffeur

Le chauffeur utilise l'application Android. Il peut démarrer un trajet, envoyer des mises à jour telles que départ, retard ou arrivée, transmettre des coordonnées GPS et créer des notifications d'approche d'arrêt. Dans la version de démonstration, le parcours chauffeur est volontairement direct afin d'être testé rapidement.

1.3.3 Parent

Le parent utilise également l'application Android, mais avec une interface différente. Il peut consulter l'enfant associé, le bus, le trajet, l'état actuel du bus, la localisation et l'historique des notifications. Cette séparation montre comment une même application mobile peut servir deux rôles sans exposer des actions inutiles.

1.4 Besoins Fonctionnels

Le système doit fournir les fonctionnalités suivantes :

- Permettre l'authentification avec un nom d'utilisateur et un mot de passe.
- Protéger les fonctionnalités selon le rôle et la plateforme de l'utilisateur.
- Permettre à l'administrateur de gérer les élèves.
- Permettre à l'administrateur de gérer les bus.
- Permettre à l'administrateur de gérer les trajets.
- Permettre à l'administrateur de gérer les comptes chauffeurs et parents.
- Permettre à l'administrateur de consulter les notifications.
- Permettre au chauffeur de démarrer un trajet.
- Permettre au chauffeur de mettre à jour l'état du bus : départ, retard et arrivée.
- Permettre au chauffeur d'envoyer une position GPS manuelle ou actuelle.
- Permettre au chauffeur d'envoyer une notification d'approche d'arrêt.
- Permettre au parent de consulter les informations de transport de son enfant.
- Permettre au parent de consulter et filtrer les notifications.
- Emettre des événements en temps réel lors des mises à jour des bus ou des notifications.
- Fournir une opération de réinitialisation des données de démonstration.

1.5 Besoins Non Fonctionnels

Le projet prend aussi en compte plusieurs besoins non fonctionnels :

- **Utilisabilité** : les interfaces web et mobile doivent être directes et compréhensibles.
- **Maintenabilité** : le code doit être organisé en modules clairs.
- **Sécurité** : l'accès aux fonctionnalités doit être protégé par authentification et autorisation.
- **Fiabilité** : la validation backend et la gestion des erreurs doivent limiter les données incorrectes.
- **Portabilité** : Docker Compose doit faciliter l'exécution locale sur une machine d'évaluation.
- **Extensibilité** : le système doit pouvoir évoluer vers les notifications push, l'historique ou une gestion avancée des utilisateurs.

1.6 Contraintes

Le projet est développé comme une démonstration académique. Certaines décisions ont été prises pour garder un périmètre réaliste :

- Les utilisateurs de démonstration sont gérés dans le code pour l'authentification.
- L'application mobile cible uniquement Android.
- Le tableau de bord web est réservé au rôle administrateur.
- La base de données se concentre sur les principales tables du domaine transport.
- Le projet est conçu pour une exécution locale avec Docker Compose.

1.7 Conclusion

Ces besoins montrent que SafeBus doit coordonner plusieurs acteurs sans devenir inutilement complexe. Le projet conserve donc un modèle de domaine réduit, mais le relié à des fonctionnalités full-stack réalistes comme la vérification des rôles, les mises à jour en temps réel, la consommation d'API mobile et l'exécution avec Docker.

Chapitre 2

Notions Techniques Preliminaires

2.1 Introduction

Ce chapitre présente les principales technologies et notions utilisées dans SafeBus. Ces éléments permettent de comprendre pourquoi le système est organisé autour d'une API backend, d'un tableau de bord web, d'une application mobile, d'une base de données relationnelle et d'une communication en temps réel.

2.2 API REST

Une API REST est une interface de communication qui expose des ressources à travers des endpoints HTTP. Dans SafeBus, le backend fournit des endpoints pour l'authentification, les élèves, les bus, les trajets, les notifications et les actions du chauffeur. Le tableau de bord web et l'application mobile communiquent avec le backend par des requêtes HTTP.

REST convient à ce projet car il est simple, largement utilisé et compatible avec les clients web et mobile. Chaque fonctionnalité peut être représentée par un endpoint clair comme `/students`, `/buses` ou `/driver/update-location`.

2.3 Authentification par JSON Web Token

JSON Web Token, ou JWT, est utilisé pour authentifier les utilisateurs après la connexion. Lorsqu'un utilisateur fournit des identifiants valides, le backend retourne un token. Le client envoie ensuite ce token dans l'en-tête `Authorization` des requêtes protégées.

Ce mécanisme permet au backend d'identifier le rôle de l'utilisateur et de vérifier si l'opération demandée est autorisée. Par exemple, l'administrateur accède aux fonctions de gestion du tableau de bord, tandis que les actions chauffeur sont réservées au rôle chauffeur.

2.4 Communication en Temps Réel

Les requêtes HTTP traditionnelles permettent de récupérer ou modifier des données, mais elles ne notifient pas automatiquement les clients lorsqu'une nouvelle information apparaît. SafeBus utilise Socket.IO pour gérer des événements en temps réel. Quand l'état ou la position d'un bus change, le backend émet un événement que le tableau de bord peut recevoir.

Cette communication est importante dans un système de transport, car l'état et la position du bus sont des données dynamiques. Le tableau de bord ne doit pas dépendre uniquement d'un rafraîchissement manuel.

2.5 Base de Données Relationnelle

SafeBus utilise MySQL comme base de données relationnelle. La base stocke les principales données de transport : élèves, bus, trajets et notifications. Les tables SQL sont adaptées parce que ces données possèdent des relations structurées, par exemple un élève associé à un bus et à un trajet.

2.6 Tableau de Bord Web

L'interface administrateur est implémentée sous forme d'application React. React permet de diviser le tableau de bord en composants et pages réutilisables. Vite est utilisé comme outil de développement et de build, car il offre un démarrage rapide et une configuration simple.

2.7 Application Android Native

La partie mobile de SafeBus est implémentée sous Android natif avec Kotlin. L'application utilise Retrofit pour communiquer avec l'API backend, ViewModel et LiveData pour gérer l'état des écrans, RecyclerView pour afficher les listes de notifications et SharedPreferences pour stocker localement la session.

2.8 Docker Compose

Docker Compose est utilisé pour démarrer la base de données, le backend et le tableau de bord web avec une seule commande. Cela rend le projet plus simple à exécuter et à présenter, car l'évaluateur n'a pas besoin de configurer chaque service séparément.

2.9 Conclusion

Les technologies utilisées dans SafeBus forment une base cohérente pour une solution full-stack. REST, JWT, Socket.IO, MySQL, React, Android Kotlin et Docker Compose permettent de couvrir les principaux besoins du projet tout en gardant une structure compréhensible.

Chapitre 3

Travaux Connexes et Choix Technologiques

3.1 Introduction

SafeBus appartient à la catégorie générale des systèmes de suivi de transport et de supervision de flotte. Ce chapitre situe brièvement le projet dans ce domaine et explique les choix technologiques effectués pour l'implémentation académique.

3.2 Systèmes de Suivi de Transport

Les systèmes modernes de suivi de transport fournissent généralement trois capacités principales : le partage de localisation, la communication de l'état et la notification des utilisateurs. Dans le transport public, ces fonctions aident les passagers à estimer les arrivées. Dans la logistique, elles aident les entreprises à superviser les livraisons. Dans le transport scolaire, elles prennent une dimension plus sensible, car les parents et l'administration ont besoin d'informations fiables sur le déplacement quotidien des élèves.

SafeBus ne reproduit pas toutes les fonctionnalités des plateformes avancées de gestion de flotte. Il conserve les fonctions les plus utiles pour un premier prototype scolaire :

- Gestion des bus et des trajets.
- Affectation des élèves aux bus et aux trajets.
- Mise à jour de l'état par le chauffeur.
- Mise à jour de la position GPS.
- Notifications aux parents.
- Supervision par l'administrateur.

3.3 Choix des Technologies Backend

Node.js et Express ont été choisis pour le backend parce qu'ils permettent de développer rapidement des API REST et s'intègrent naturellement avec une application web JavaScript. Express facilite aussi l'organisation du backend en routes, contrôleurs, services, validateurs, modèles et middlewares.

MySQL a été choisi comme base de données parce que les données du projet sont structurées et relationnelles. Les élèves, bus, trajets et notifications peuvent être représentés clairement par des tables et des relations. MySQL est aussi largement utilisé et facile à lancer avec Docker.

JWT a été choisi pour l'authentification car il est léger et adapté à la protection d'une API stateless. Socket.IO a été choisi pour la communication en temps réel parce qu'il simplifie les événements entre le backend et le tableau de bord.

3.4 Choix des Technologies Web

React a été choisi pour le tableau de bord parce que l'interface contient des éléments répétés comme les tableaux, badges, en-têtes de page, états de chargement, états vides, notices et sections de mise en page. Le développement par composants permet de garder ces éléments cohérents entre les pages.

Vite a été choisi pour offrir un environnement de développement rapide et un build simple. Axios est utilisé pour les requêtes HTTP, React Router pour la navigation et Socket.IO Client pour les mises à jour en temps réel.

3.5 Choix des Technologies Mobiles

L'application mobile a été implémentée nativement sur Android avec Kotlin. Ce choix donne accès aux fonctionnalités Android comme la localisation, le stockage local, les intents pour ouvrir une carte et les composants d'interface natifs.

Retrofit est utilisé pour communiquer avec l'API backend. ViewModel et LiveData permettent de séparer l'état des écrans du cycle de vie des activités. RecyclerView affiche efficacement les listes de notifications, et SharedPreferences conserve le token et les données de session.

3.6 Choix de Déploiement

Docker Compose a été choisi pour simplifier l'exécution locale. Au lieu d'installer et configurer manuellement MySQL, le backend et le tableau de bord web, le projet peut être lancé avec :

```
1 docker compose up --build
```

Cette approche est utile pour l'évaluation et la démonstration, car elle réduit les erreurs de configuration et rend l'environnement plus reproductible.

3.7 Conclusion

Les choix technologiques de SafeBus cherchent un équilibre entre simplicité, maintenabilité et valeur démonstrative. La stack choisie est suffisamment réaliste pour montrer un projet full-stack complet, tout en restant maîtrisable dans un contexte académique.

Chapitre 4

Analyse et Conception

4.1 Introduction

Ce chapitre présente les choix de conception de SafeBus. Il explique comment le système est divisé entre clients, services backend, tables de base de données, workflows et règles de sécurité.

4.2 Architecture Globale

SafeBus suit une architecture client-serveur. L'API backend est le point central du système : elle reçoit les requêtes du tableau de bord et de l'application mobile, applique les règles métier, stocke les données dans MySQL et émet des événements en temps réel lorsque des informations importantes changent.

```
graph LR; A[Tableau de bord administrateur] --> B[API Backend]; B --> C[Base MySQL]; D[Application Android chauffeur] --> B; E[Application Android parent] --> B; B --> F[événements Socket.IO]; F --> G[Tableau de bord];
```

Tableau de bord administrateur → API Backend → Base MySQL
Application Android chauffeur → API Backend → Base MySQL
Application Android parent → API Backend → Base MySQL
API Backend → événements Socket.IO → Tableau de bord

Figure 4.1 : Architecture globale de SafeBus

Cette structure garde les clients relativement simples. Le tableau de bord se concentre sur les écrans d'administration, et l'application Android se concentre sur les actions parent et chauffeur. Le backend conserve les règles de manière uniforme pour les deux clients.

4.3 Rôles et Permissions

4.4 Conception en Couches du Backend

Le code source backend est organisé en couches correspondant aux responsabilités visibles dans le dépôt :

- **Routes** : définissent les endpoints API et les lient aux contrôleurs.
- **Contrôleurs** : gèrent les requêtes et réponses HTTP.
- **Services** : contiennent la logique métier, comme la création des notifications.

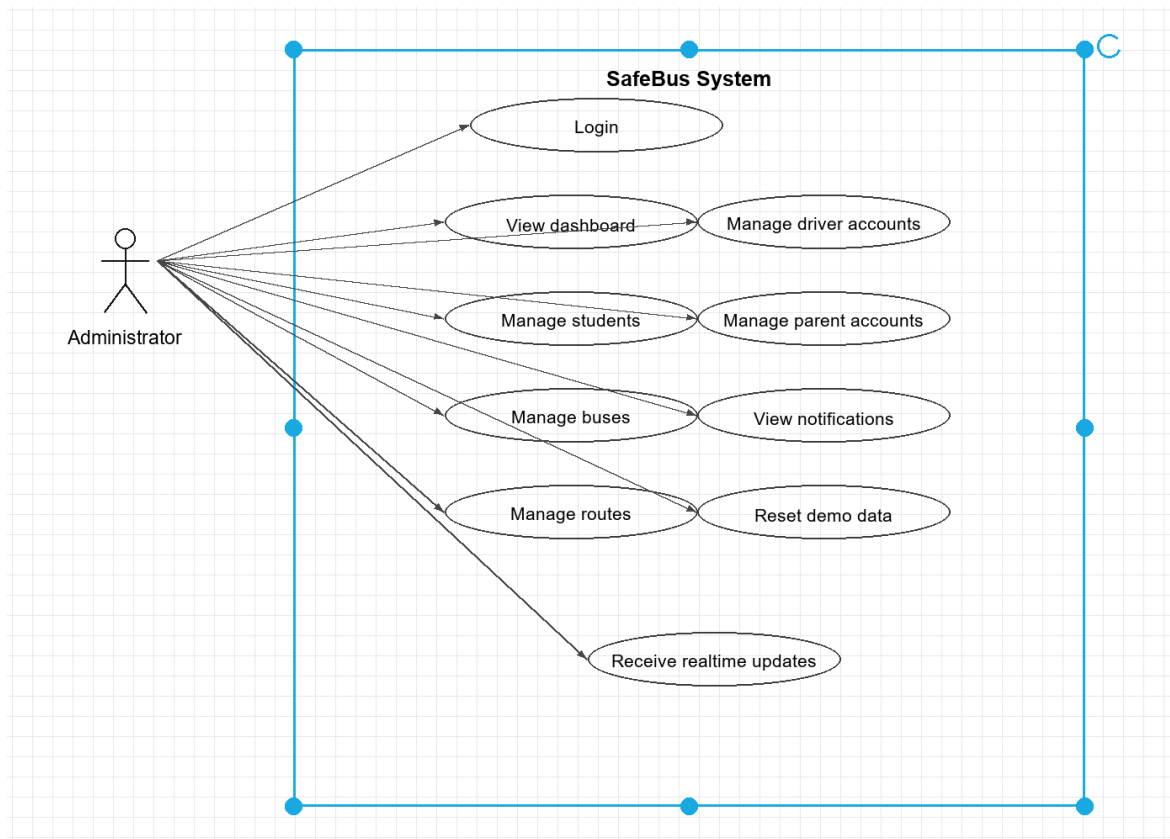


Figure 4.2 : Diagramme de cas d'utilisation administrateur

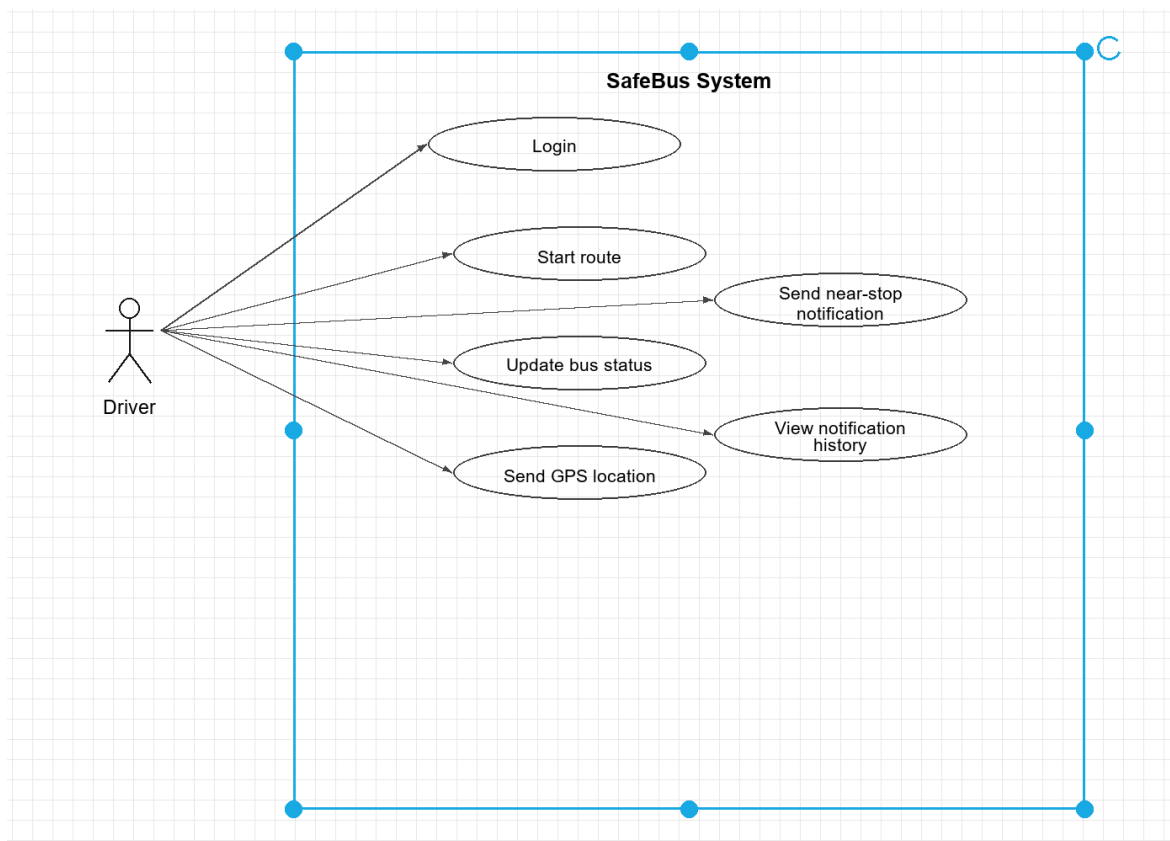


Figure 4.3 : Diagramme de cas d'utilisation chauffeur

Rôle	Plateforme	Permissions principales
Administrateur	Tableau de bord web	Gérer les élèves, bus, trajets, comptes chauffeurs, comptes parents et notifications.
Chauffeur	Application Android	Démarrer le trajet, mettre à jour l'état du bus, envoyer la position GPS et créer des notifications d'approche.
Parent	Application Android	Consulter les données de l'enfant, l'état du bus, la localisation et les notifications.

Tableau 4.1 : Rôles et permissions

- **Modèles** : interagissent avec la base de données.
- **Middlewares** : gèrent l'authentification, la validation et les erreurs.
- **Validateurs** : définissent les règles de validation des entrées.
- **Configuration** : regroupe l'environnement, la base de données et le temps réel.

Cette organisation rend le backend plus facile à modifier. Par exemple, une règle de validation peut être changée dans le dossier des validateurs sans réécrire le contrôleur, et la logique de notification peut évoluer dans la couche service sans modifier les définitions de routes.

4.5 Modèle de Données

La base de données est initialisée à partir de `backend/database/schéma.sql` et `backend/database/seed.sql`. Le schéma contient quatre tables principales : `buses`, `routes`, `students` et `parent_notifications`. Ce modèle réduit suffit à supporter les workflows de démonstration tout en gardant les relations compréhensibles.

Entité	Rôle
Buses	Stocke l'identité du bus, les informations chauffeur, l'état actuel et les coordonnées GPS.
Routes	Stocke le nom du trajet, les arrêts, les horaires et le bus assigné.
Students	Stocke l'identité de l'élève, les informations parent et l'affectation à un bus et à un trajet.
Parent notifications	Stocke le type de notification, le message, l'élève concerne et la date de création.

Tableau 4.2 : Principales entités de données

4.6 Workflows Principaux

4.6.1 Workflow d'Authentification

L'utilisateur envoie un nom d'utilisateur et un mot de passe au backend. Si les identifiants sont valides, le backend retourne un token JWT et les informations de rôle. Le client stocke le token et l'envoie dans les futures requêtes protégées. Le client web accepte le rôle administrateur, tandis que le client mobile oriente les rôles parent et chauffeur vers les écrans correspondants.

4.6.2 Workflow de Gestion Administrateur

L'administrateur utilise le tableau de bord web pour créer, modifier et supprimer les données de transport. Chaque opération traverse la même chaîne backend : vérification du token, traitement par le contrôleur, exécution du service, requête du modèle et réponse de la base de données. Ce parcours garde le dashboard simple tout en protégeant la cohérence des élèves, des bus et des trajets.

4.6.3 Workflow de Mise à Jour du Chauffeur

Le chauffeur envoie une mise à jour comme départ, retard ou arrivée. Le backend vérifie le rôle chauffeur, met à jour l'état du bus, crée des notifications pour les élèves concernés, enregistre le changement et émet des événements en temps réel. Ce workflow relie une action chauffeur à l'information des parents et à la visibilité du tableau de bord.

4.6.4 Workflow de Position GPS

Le chauffeur envoie une latitude et une longitude, soit manuellement, soit depuis la localisation du téléphone. Le backend valide les coordonnées, met à jour l'enregistrement du bus et émet des événements en temps réel pour permettre au tableau de bord d'afficher la nouvelle position.

4.6.5 Workflow de Notification Parent

Lorsqu'une action chauffeur crée une notification, le parent peut la récupérer depuis l'application mobile. La liste des notifications peut aussi être filtrée dans le système, ce qui rend la fonctionnalité utile pour la consultation parent et la supervision administrateur.

4.7 Conception de la Sécurité

SafeBus utilise JWT pour l'authentification et une autorisation basée sur les rôles. Le backend vérifie le token avant d'autoriser les actions protégées. Cela empêche un parent d'accéder aux fonctionnalités administrateur et empêche un compte dashboard d'exécuter des opérations réservées au chauffeur.

Pour une version de production, ce modèle devrait être complété par une vraie table utilisateurs, le hachage systématique des mots de passe, une meilleure gestion des secrets, des refresh tokens et une configuration d'environnement plus stricte.

4.8 Conclusion

La conception de SafeBus suit une structure client-serveur modulaire. Le backend centralise les données et les règles métier, tandis que les clients web et mobile se concentrent sur les interfaces propres à chaque rôle. Cette conception fournit une base claire pour l'implémentation et laisse la possibilité d'ajouter des fonctionnalités futures.

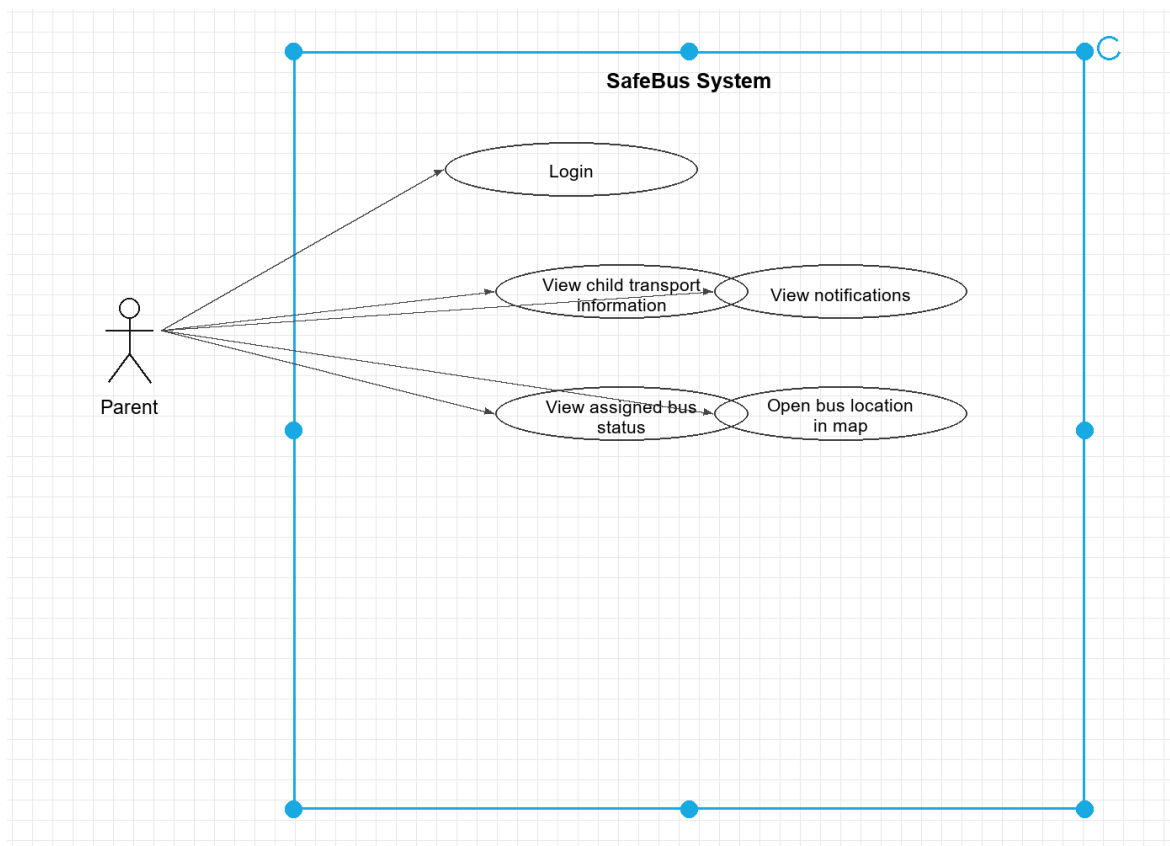


Figure 4.4 : Diagramme de cas d'utilisation parent

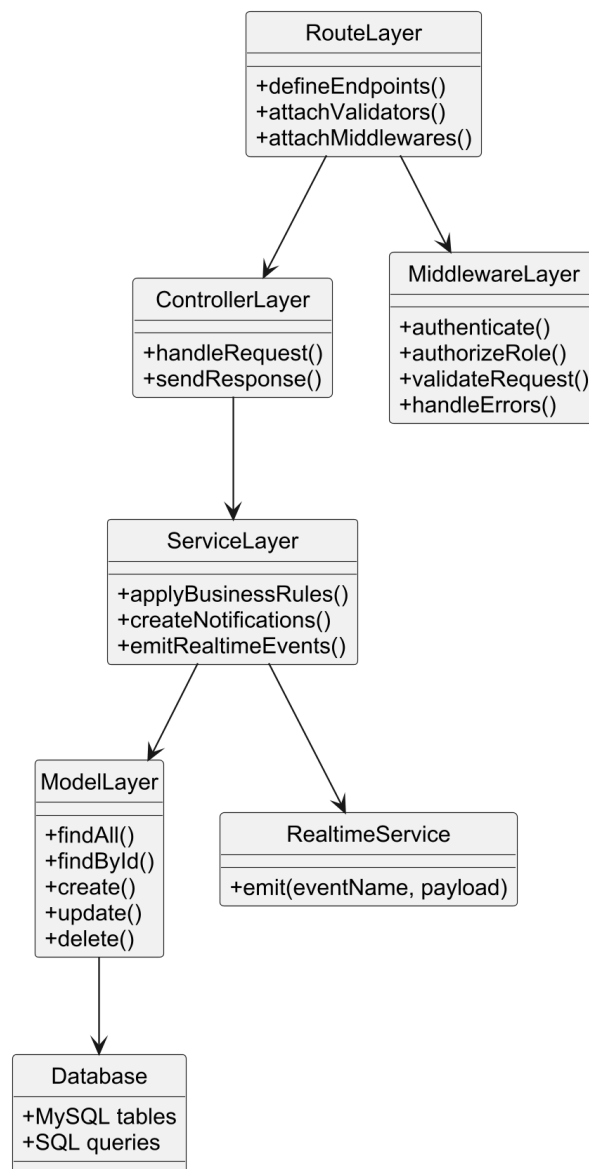


Figure 4.5 : Diagramme de classes de la structure backend

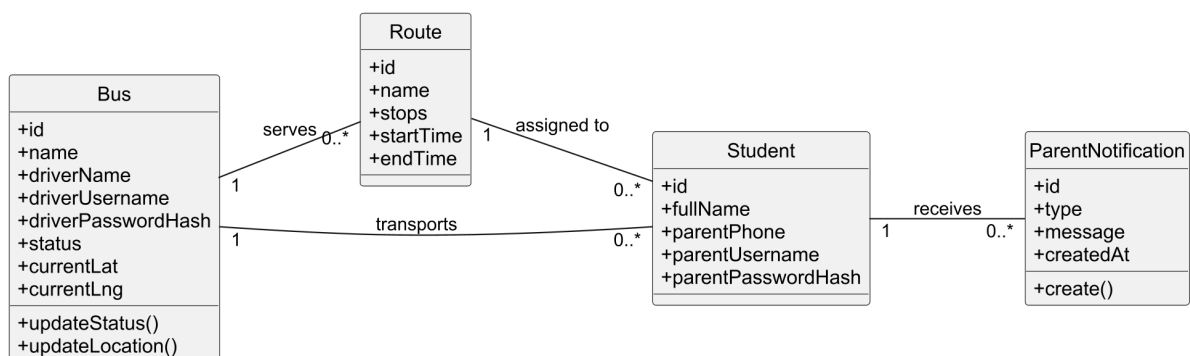


Figure 4.6 : Diagramme de classes du modèle de domaine SafeBus

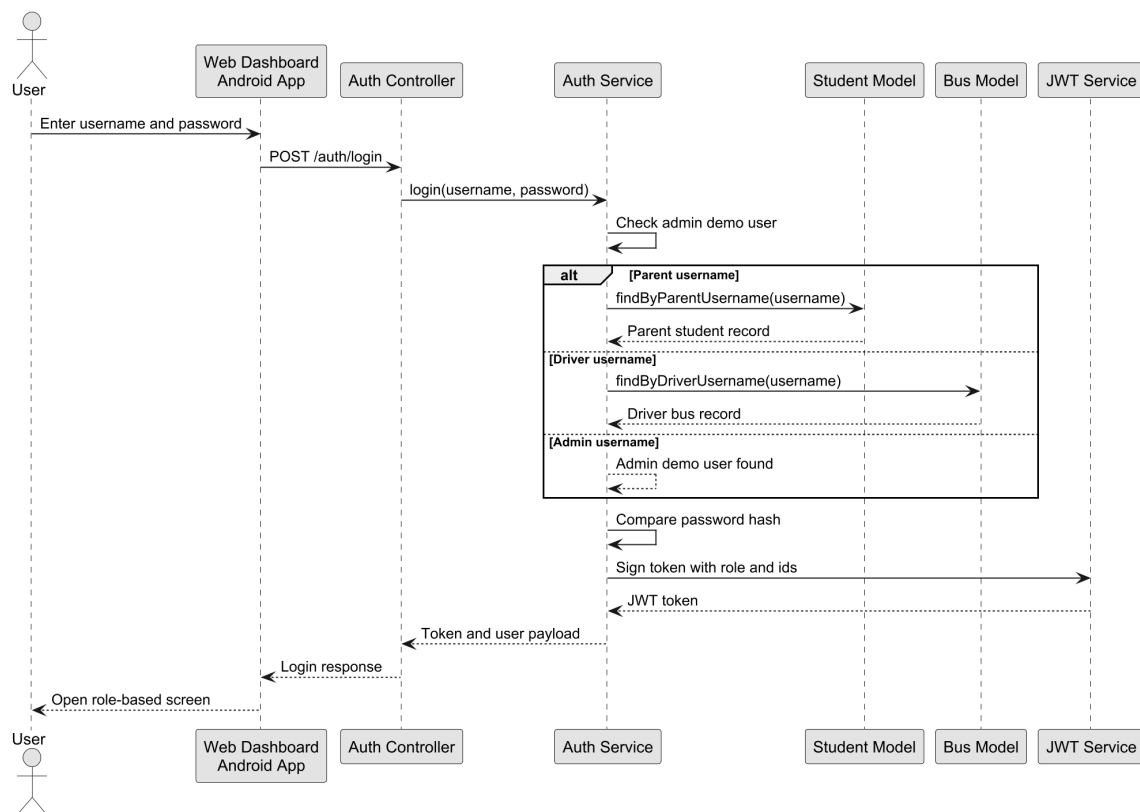


Figure 4.7 : Diagramme de sequence de l'authentification

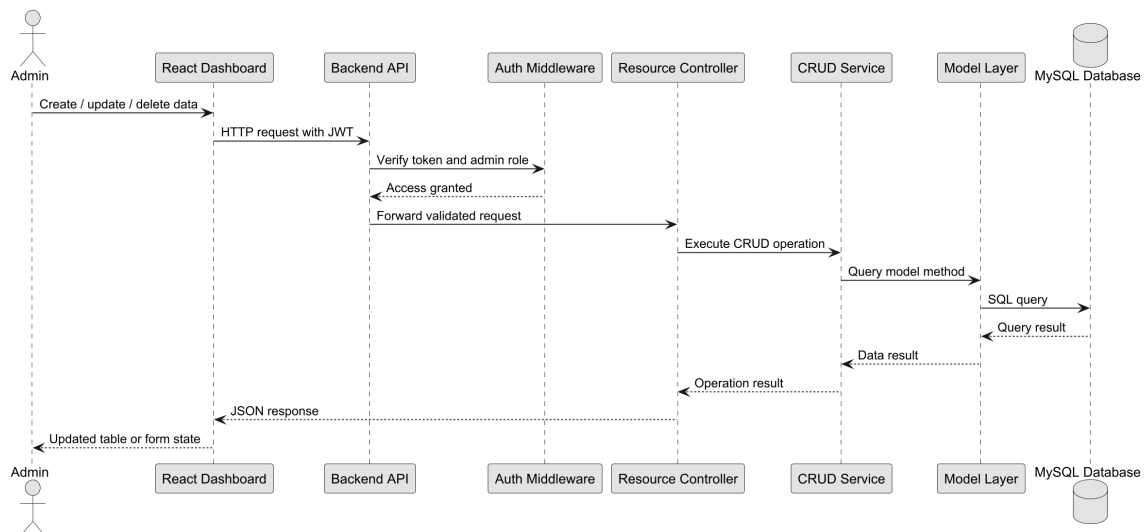


Figure 4.8 : Diagramme de sequence de la gestion des données administrateur

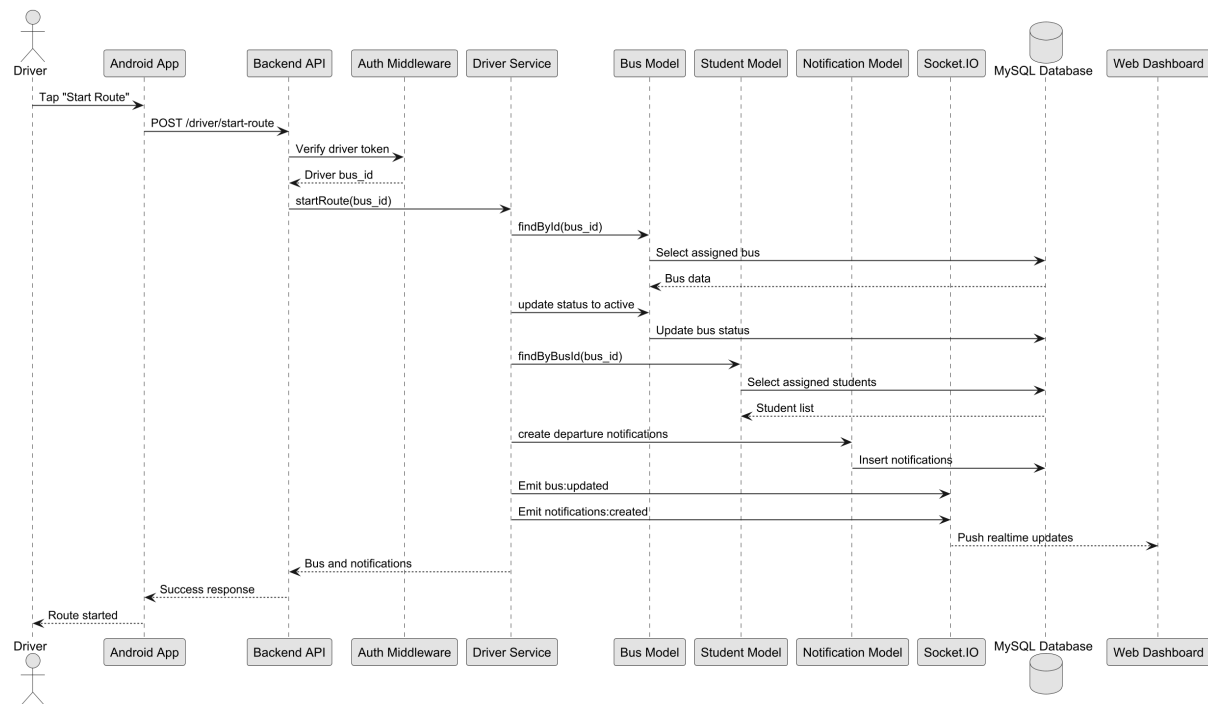


Figure 4.9 : Diagramme de sequence du démarrage de trajet chauffeur

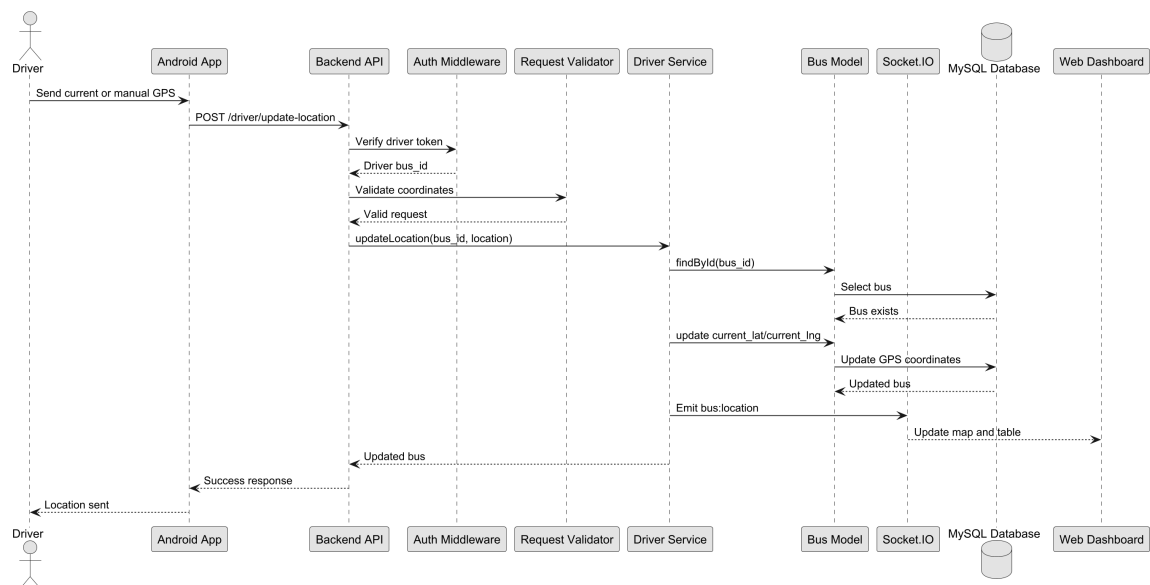


Figure 4.10 : Diagramme de sequence de la mise à jour GPS chauffeur

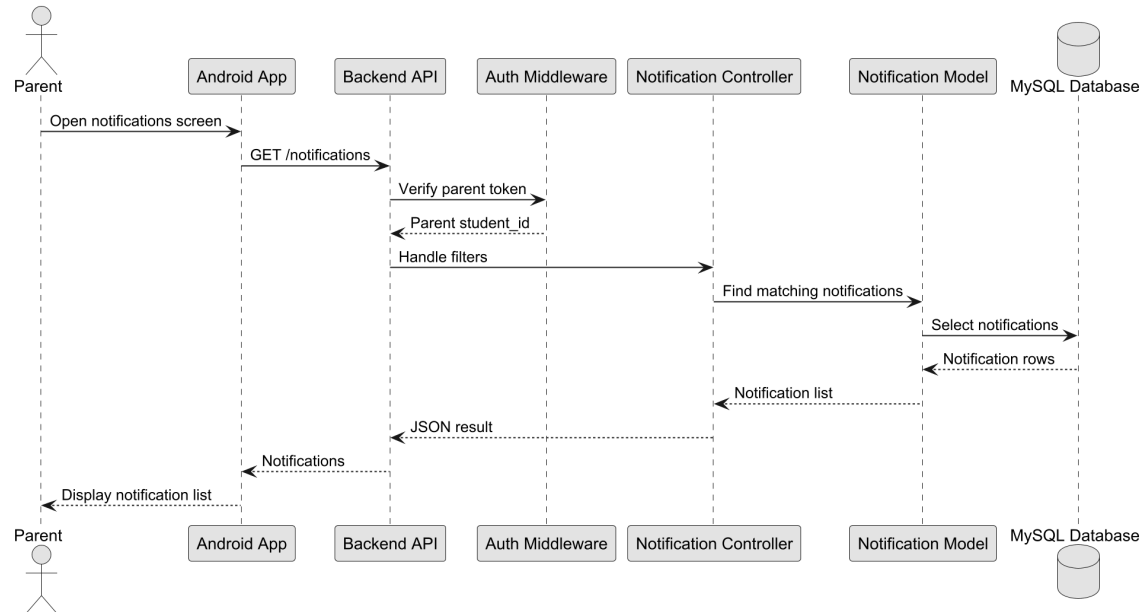


Figure 4.11 : Diagramme de sequence de consultation des notifications parent

Chapitre 5

Réalisation

5.1 Introduction

Ce chapitre décrit comment la conception de SafeBus a été transformée en application fonctionnelle. Il couvre l'API backend, la couche temps réel, le tableau de bord React, l'application Android, les scripts de base de données, la configuration de déploiement et la validation manuelle.

5.2 Réalisation du Backend

Le backend est implémenté avec Node.js et Express. Le serveur démarre l'application HTTP et attache Socket.IO pour la communication en temps réel. L'application utilise des middlewares pour CORS, le parsing JSON, les logs de requêtes, l'enregistrement des routes, la validation, l'authentification et la gestion centralisée des erreurs.

5.2.1 Structure Backend

```
1 backend/  
2   src/  
3     config/  
4     controllers/  
5     middlewares/  
6     models/  
7     routes/  
8     services/  
9     validators/  
10    utils/
```

Le backend suit une séparation des responsabilités. Les routes exposent les endpoints, les contrôleurs gèrent les entrées et sorties HTTP, les services implémentent la logique métier, les modèles communiquent avec MySQL et les validateurs protègent l'API contre les données invalides avant qu'elles arrivent à la couche service.

5.2.2 Authentification

L'authentification est gérée par l'endpoint `/auth/login`. Les utilisateurs de démonstration sont définis dans le code pour simplifier l'exécution pendant l'évaluation. Après une connexion réussie, le backend retourne un token JWT. Les routes protégées utilisent un middleware d'authentification pour vérifier le token et le rôle attendu.

5.2.3 Principaux Endpoints API

Utilisateur	Mot de passe	Rôle
admin	admin123	admin
driver	driver123	driver
parent	parent123	parent

Tableau 5.1 : Utilisateurs de démonstration

Méthode	Endpoint	Description
GET	/health	Vérifier l'état de l'API.
POST	/auth/login	Authentifier un utilisateur.
GET/POST/PUT/DELETE	/students	Gérer les élèves.
GET/POST/PUT/DELETE	/buses	Gérer les bus.
GET/POST/PUT/DELETE	/trajets	Gérer les trajets.
GET/POST	/notifications	Consulter ou créer des notifications.
POST	/driver/start-route	Démarrer le trajet du chauffeur.
POST	/driver/update-status	Mettre à jour l'état du bus.
POST	/driver/update-location	Mettre à jour la position GPS du bus.
POST	/driver/near-stop	Créer une notification d'approche d'arrêt.
POST	/demo/reset	Réinitialiser les données de démonstration.

Tableau 5.2 : Principaux endpoints API

5.3 Réalisation du Temps Réel

Socket.IO est utilisé pour émettre des événements lorsque des données importantes changent. Les principaux événements sont :

- `bus:updated` : émis lorsqu'un état ou une information du bus change.
- `bus:location` : émis lorsqu'un chauffeur envoie des coordonnées GPS.
- `notifications:created` : émis lorsque des notifications sont générées.
- `demo:reset` : émis lorsque les données de démonstration sont réinitialisées.

Ces événements permettent au tableau de bord de réagir sans rafraîchissement manuel. Par exemple, lorsqu'un chauffeur envoie une position GPS, le backend met à jour le bus et émet un événement consommable par le client web.

5.4 Réalisation du Tableau de Bord Web

Le tableau de bord web est implémenté avec React et Vite. Il est dédié au rôle administrateur. L'interface est divisée en composants réutilisables pour la mise en page, les en-têtes, les badges d'état, les chargements, les états vides, les notices, les badges de notification et l'affichage de carte.

5.4.1 Structure Web

```

1 web/
2   src/
3     api/
4     components/
5     pages/

```

Le dossier `api` contient le client Axios et la connexion temps réel. Le dossier `components` contient les éléments réutilisables de l'interface. Le dossier `pages` contient les écrans principaux : connexion, dashboard, bus, élèves, trajets, notifications, comptes chauffeurs et comptes parents.

5.4.2 Fonctionnalités du Dashboard

L'administrateur peut se connecter, consulter les informations du dashboard, gérer les entités de transport, voir les notifications et consulter les informations de localisation des bus. Le tableau de bord se connecte au backend à l'adresse :

```
1 http://localhost:5000
```

Le tableau de bord fonctionne localement à l'adresse :

```
1 http://localhost:3000
```

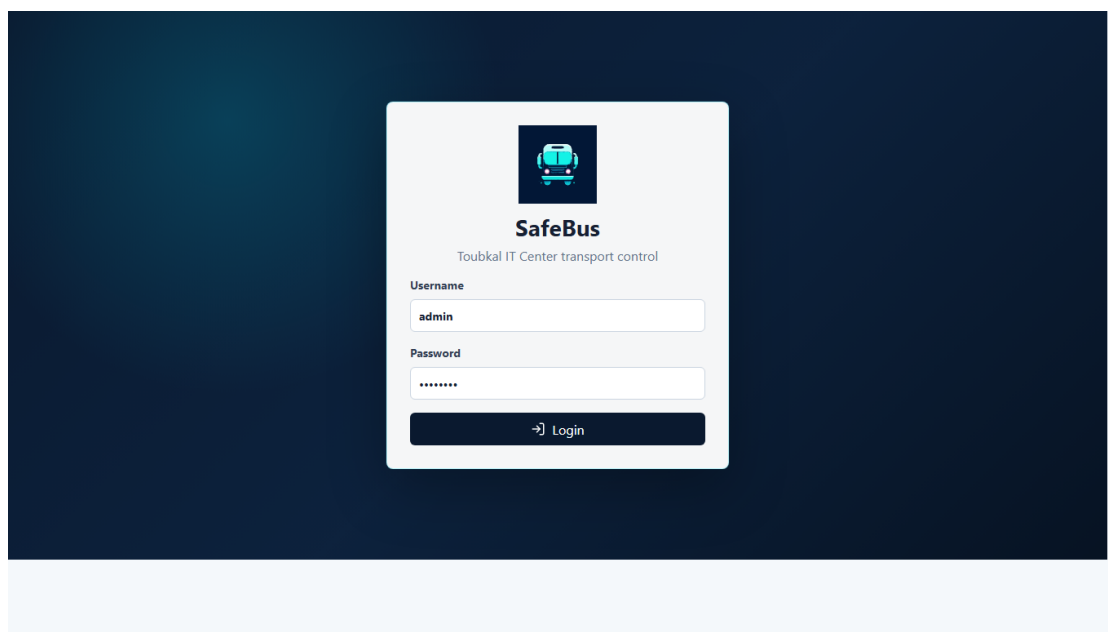


Figure 5.1 : écran de connexion administrateur SafeBus

Le dashboard a été pensé pour une supervision rapide plutôt que pour une navigation decorative. L'administrateur voit directement les bus actifs, les retards, les trajets terminés et les élèves inscrits, puis il peut accéder aux tableaux de gestion. Le panneau de notification de test facilite aussi la démonstration du comportement des notifications Android pendant l'évaluation.

5.5 Réalisation de l'Application Android

L'application Android est implémentée avec Kotlin. Elle supporte deux rôles : parent et chauffeur. L'application bloque volontairement l'ouverture d'un écran mobile pour le rôle administrateur, car ce rôle appartient au tableau de bord web.

5.5.1 Structure Mobile

```
1 mobile/app/src/main/java/com/schooltransport/mobile/  
2   api/  
3   data/  
4   repository/  
5   ui/  
6   viewmodel/
```

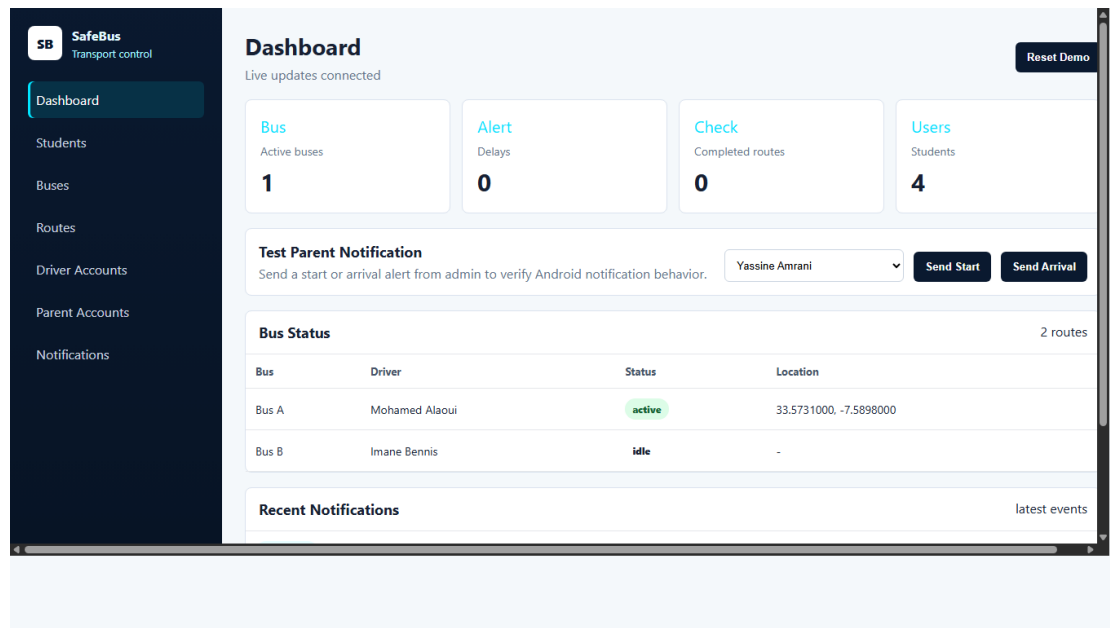



Figure 5.2 : Dashboard administrateur avec les indicateurs de transport

Le package `api` contient la configuration Retrofit et les définitions d'endpoints. Le package `data` contient les modèles et la gestion de session. Le package `repository` centralise les appels API. Le package `ui` contient les activités, adaptateurs et aides de notification. Le package `viewmodel` gère la logique des écrans et sépare l'état de l'interface des activités.

5.5.2 Fonctionnalités Parent

Le parent peut consulter les informations de l'enfant, le bus assigné, le nom du trajet, l'état en direct et les notifications. Il peut aussi ouvrir la position du bus dans une application de carte, rafraîchir les données, filtrer les notifications et se déconnecter.

5.5.3 Fonctionnalités Chauffeur

Le chauffeur peut démarrer un trajet, envoyer les états départ, retard et arrivée, envoyer des coordonnées GPS manuelles, transmettre la position actuelle du téléphone, créer des notifications d'approche, consulter l'historique des notifications, ouvrir la position du bus dans une carte, rafraîchir les données et se déconnecter.

Les écrans Android gardent les actions proches du rôle de l'utilisateur. L'interface chauffeur met en avant les actions de trajet et l'envoi GPS, tandis que l'interface parent met en avant l'enfant, le bus assigné, l'état du trajet, l'ouverture de carte et l'historique des notifications. Cette séparation évite d'exposer les opérations d'administration dans l'application mobile.

5.6 Réalisation de la Base de Données

La base de données est initialisée automatiquement par Docker Compose à l'aide de scripts SQL. Le schéma définit les tables `buses`, `routes`, `students` et `parent_notifications`. Le script de seed insère les données de démonstration pour les tests et la présentation.

Le conteneur MySQL expose le port 3306. Le backend se connecte à la base de données à l'aide des variables d'environnement définies dans `docker-compose.yml`.

8:41



Figure 5.3 : écran de connexion Android

5.7 Réalisation du Déploiement

Docker Compose démarre trois services principaux :

- **mysql** : base de données MySQL 8.
- **backend** : serveur API Node.js sur le port 5000.
- **web** : tableau de bord React sur le port 3000.

La commande principale d'exécution est :

```
1 docker compose up --build
```

5.8 Tests et Validation

La validation manuelle a été organisée autour des principaux parcours utilisateur, car le projet est avant tout un prototype académique :

- Connexion administrateur et accès au tableau de bord.
- Operations CRUD sur les élèves, bus et trajets.
- Démarrage du trajet et mises à jour chauffeur.
- Mise à jour GPS par le chauffeur.
- Affichage des notifications côté parent.

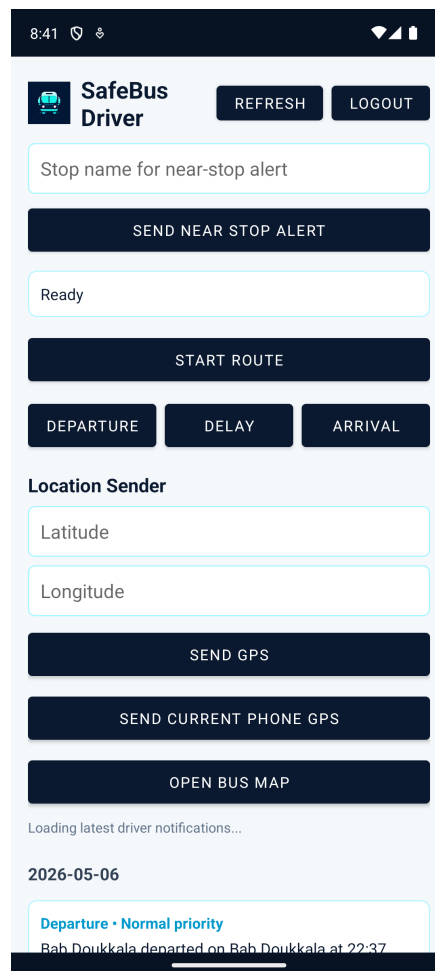


Figure 5.4 : écran chauffeur capture depuis l'émulateur

- Mises à jour en temps réel du dashboard après les actions chauffeur.
- Réinitialisation des données de démonstration.

5.9 Conclusion

L'implémentation de SafeBus montré l'intégration du backend, du web, du mobile, de la base de données et du temps réel dans un seul projet cohérent. Le résultat est un prototype fonctionnel pouvant être exécuté localement et utilisé pour démontrer les principaux parcours de suivi du transport scolaire.

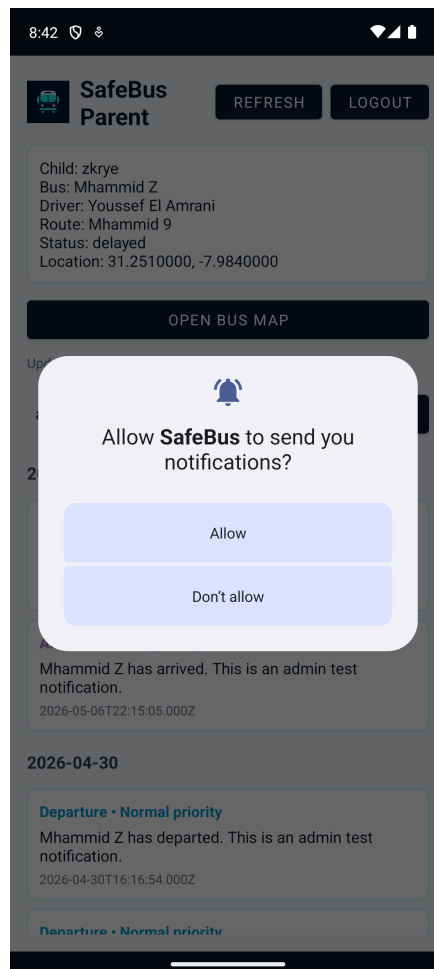


Figure 5.5 : écran parent capture depuis l'émulateur

Chapitre 6

Conclusion et Perspectives

6.1 Conclusion Générale

SafeBus a été développé comme un système full-stack de suivi du transport scolaire qui connecte l'administration, les chauffeurs et les parents à travers une plateforme cohérente. Le projet répond à un problème de communication concret : les informations de transport scolaire sont souvent transmises manuellement, alors que les parents et l'administration ont besoin de mises à jour plus claires et plus rapides.

Le projet met en pratique plusieurs concepts importants du génie logiciel. Il utilise des API REST pour la communication, JWT pour l'authentification, l'autorisation par rôle pour le contrôle d'accès, MySQL pour la persistance structurée, Socket.IO pour les événements en temps réel, React pour le tableau de bord web, Kotlin pour l'application Android et Docker Compose pour l'exécution locale.

Le prototype final supporte les principaux workflows attendus. L'administrateur peut gérer les données de transport depuis le tableau de bord. Le chauffeur peut démarrer un trajet, mettre à jour l'état, envoyer une position et créer des notifications. Le parent peut consulter l'état du bus associé et l'historique des notifications. Le backend centralise la logique métier et envoie des mises à jour en temps réel lorsque l'état du transport change.

6.2 Resultats du Projet

Les principaux résultats du projet sont :

- Une API backend fonctionnelle avec authentification, validation, opérations CRUD, actions chauffeur et événements temps réel.
- Un tableau de bord administrateur React avec pages de gestion et mises à jour en direct.
- Une application Android native pour les usages parent et chauffeur.
- Un schéma MySQL et des données initiales pour des démonstrations répétables.
- Un environnement Docker Compose pour exécuter le web, le backend et la base de données.
- Une documentation et une structure de dépôt préparées pour l'évaluation et la maintenance future.

6.3 Limites

Bien que le projet soit fonctionnel, il reste un prototype académique. Les principales limites sont :

- L'authentification de démonstration est simplifiée et doit être remplacée par une gestion complète des utilisateurs.
- Les notifications push ne sont pas encore implémentées.

- L'application mobile cible uniquement Android.
- Les tests automatisés ne sont pas encore inclus.
- Le déploiement cloud n'est pas encore configuré.
- L'optimisation avancée des trajets et la prédiction d'heure d'arrivée ne sont pas intégrées.

6.4 Perspectives

Plusieurs améliorations peuvent être ajoutées dans les versions futures :

- Ajouter une gestion production des utilisateurs avec hachage des mots de passe et récupération de compte.
- Ajouter les notifications push pour les parents.
- Ajouter l'historique des trajets et les rapports d'activité chauffeur.
- Ajouter un suivi cartographique en direct directement dans l'application mobile.
- Ajouter le support de plusieurs enfants pour un même parent.
- Ajouter des tests automatisés d'API et des tests end-to-end du dashboard.
- Ajouter une pipeline CI/CD.
- Déployer le système sur un serveur cloud.
- Renforcer la configuration de sécurité pour une utilisation en production.

6.5 Remarques Finales

SafeBus montré comment le développement web et mobile peut être combiné pour traiter un problème pratique de communication. Le projet démontré aussi le chemin complet allant des besoins et de l'architecture jusqu'à l'implémentation, l'intégration, l'exécution locale et la documentation.

Chapitre 7

Contribution Personnelle et Acquis

7.1 Introduction

Ce chapitre résumé la contribution personnelle réalisée pendant le projet SafeBus ainsi que les compétences développées. Le projet a demandé de l'analyse, de la conception, du développement backend, du développement frontend, du développement mobile, de l'intégration, des tests et de la documentation.

7.2 Contribution Personnelle

Les principales contributions personnelles sont :

- Analyse du problème de communication et de suivi du transport scolaire.
- Définition des principaux utilisateurs et workflows.
- Conception de l'architecture du projet.
- Réalisation de l'API backend avec Express et MySQL.
- Réalisation de l'authentification et de la protection par rôle.
- Réalisation des actions chauffeur et de la génération des notifications.
- Intégration des événements en temps réel avec Socket.IO.
- Développement du tableau de bord administrateur React et de ses composants réutilisables.
- Développement de l'application Android parent et chauffeur avec Kotlin.
- Préparation de la configuration Docker Compose.
- Préparation de la documentation du projet et de la structure de livraison.

7.3 Compétences Techniques Developpees

Le projet a permis de développer des compétences pratiques dans :

- Le développement d'API REST avec Express.
- L'accès à une base de données MySQL.
- L'authentification avec JWT.
- La validation des entrées et la gestion des erreurs.
- Le développement React base sur des composants.
- Le développement Android avec Kotlin.
- La consommation d'API avec Retrofit.

- La gestion d'état avec ViewModel et LiveData.
- La communication en temps réel avec Socket.IO.
- Le déploiement local avec Docker Compose.
- L'utilisation de Git et GitHub pour le versionnement.

7.4 Difficultés Rencontrées

Plusieurs difficultés ont été rencontrées durant le projet. La première concernait l'organisation du système en modules clairs tout en gardant l'implémentation assez simple pour un prototype académique. La deuxième était la coordination de trois parties qui dépendent de la même API : backend, tableau de bord web et application mobile. Une autre difficulté était la gestion du comportement par rôle afin que chaque utilisateur accède uniquement aux fonctionnalités qui le concernent.

L'intégration des mises à jour en temps réel a aussi demandé une bonne coordination entre les événements backend et l'état du frontend. Du côté mobile, la principale difficulté concernait la gestion de session, les appels API, les actions liées à la localisation et la mise à jour propre de l'interface.

7.5 Leçons Apprises

Le projet a renforcé l'importance de planifier l'architecture avant l'implémentation. Il a aussi montré qu'un projet full-stack demandé une cohérence permanente entre la structure de base de données, les contrats API, les attentes du frontend et les workflows mobiles.

Une autre leçon importante est la valeur de la documentation. Un projet est plus facile à évaluer et à maintenir lorsque les étapes d'installation, les rôles utilisateurs, les endpoints et l'architecture sont clairement décrits.

7.6 Conclusion

SafeBus a constitué une expérience complète en développement web et mobile. Il a permis d'appliquer des connaissances théoriques à un projet pratique et de produire un prototype fonctionnel pouvant être démontré, documenté et amélioré dans le futur.

Bibliographie

- Docker Inc. (2026), 'Docker compose documentation', <https://docs.docker.com/compose/>. Official documentation for Docker Compose.
- Express.js (2026), 'Express.js documentation', <https://expressjs.com/>. Official documentation for Express web framework.
- Google (2026), 'Android developers documentation', <https://developer.android.com/>. Official documentation for Android application development.
- JetBrains (2026), 'Kotlin documentation', <https://kotlinlang.org/docs/home.html>. Official Kotlin language documentation.
- Jones, M., Bradley, J. and Sakimura, N. (2015), 'Json web token (jwt), rfc 7519', <https://www.rfc-editor.org/rfc/rfc7519>. Specification used as a reference for JWT-based authentication.
- Meta Open Source (2026), 'React documentation', <https://react.dev/>. Official documentation for React.
- OpenJS Foundation (2026), 'Node.js documentation', <https://nodejs.org/en/docs>. Official documentation for Node.js.
- Oracle (2026), 'Mysql 8.0 reference manual', <https://dev.mysql.com/doc/>. Official MySQL documentation.
- OWASP Foundation (2026), 'Authentication cheat sheet', https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html. Security guidance for authentication mechanisms.
- Socket.IO (2026), 'Socket.io documentation', <https://socket.io/docs/>. Official documentation for realtime communication with Socket.IO.
- Square (2026), 'Retrofit documentation', <https://square.github.io/retrofit/>. Official documentation for the Retrofit HTTP client used in Android.
- Tilkov, S. (2010), *REST und HTTP : Einsatz der Architektur des Web fuer Integrationsszenarien*, dpunkt.verlag. Reference on REST architecture and HTTP-based integration.
- Vite (2026), 'Vite documentation', <https://vite.dev/guide/>. Official documentation for the Vite frontend build tool.

Annexe A

Reference API

A.1 URLs de Base

```
1 API Backend : http://localhost:5000
2 Tableau de bord web : http://localhost:3000
```

A.2 Exemple de Connexion

```
1 POST /auth/login
2 Content-Type: application/json
3
4 {
5   "username": "admin",
6   "password": "admin123"
7 }
```

A.3 En-tête d'Autorisation

```
1 Authorization: Bearer <token>
```

A.4 Exemple de Position Chauffeur

```
1 POST /driver/update-location
2 Authorization: Bearer <driver-token>
3 Content-Type: application/json
4
5 {
6   "lat": 33.5731,
7   "lng": -7.5898
8 }
```

A.5 Commande Docker

```
1 docker compose up --build
```

Annexe B

Dépôt du Projet et Notes de Livraison

B.1 Dépôt GitHub

Le code source du projet est disponible à l'adresse suivante :

<https://github.com/00zkr/safebus>

B.2 Structure du Dépôt

```
1 backend/           Backend Node.js et Express
2 web/              Tableau de bord administrateur React
3 mobile/           Application mobile Android Kotlin
4 project-archive/  Documentation et notes d'archive
5 docker-compose.yml
6 README.md
```

B.3 Scénario de Demonstration Recommande

1. Démarrer le projet avec Docker Compose.
2. Se connecter au tableau de bord web comme administrateur.
3. Présenter les élèves, les bus, les trajets et les notifications.
4. Se connecter à l'application Android comme chauffeur.
5. Démarrer le trajet et envoyer un état de départ.
6. Envoyer une mise à jour GPS.
7. Montrer la mise à jour en temps réel sur le tableau de bord.
8. Se connecter à l'application Android comme parent.
9. Montrer les informations de transport de l'enfant et les notifications.

B.4 écrans Inclus dans le Rapport

Le rapport inclut les principaux écrans nécessaires pour expliquer l'application pendant la soutenance :

- écran de connexion administrateur ;
- dashboard administrateur et indicateurs de transport ;
- écran de connexion Android capture depuis l'émulateur ;

- parcours mobile chauffeur capture depuis l'émulateur ;
- parcours mobile parent capture depuis l'émulateur.

Ces écrans correspondent aux trois acteurs du projet : administrateur, chauffeur et parent. Ils suffisent pour démontrer l'authentification, la supervision, les mises à jour de trajet, l'envoi GPS et la consultation des notifications par les parents.

Annexe C

Diagrammes UML

Cette annexe présente les principaux diagrammes UML préparés pour le projet SafeBus. Le code Mermaid modifiable de ces diagrammes est disponible dans le fichier d'archive du projet :

`project-archive/UML_DIAGRAMS.md`

C.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation identifie les trois acteurs principaux du système : administrateur, chauffeur et parent. L'administrateur gère les données de transport à travers le tableau de bord web. Le chauffeur utilise l'application mobile pour démarrer les trajets, mettre à jour l'état du bus, envoyer la position GPS et envoyer les notifications d'approche. Le parent utilise l'application mobile pour consulter les informations de transport de l'enfant, l'état du bus, la position et les notifications.

C.2 Diagrammes de Sequence

Les diagrammes de sequence décrivent les interactions les plus importantes du système :

- Authentification : connexion, détection du rôle, vérification du mot de passe et génération du JWT.
- Gestion administrateur : opérations CRUD via tableau de bord et l'API backend.
- Démarrage du trajet chauffeur : mise à jour de l'état du bus, création des notifications et émission d'événement temps réel.
- Mise à jour GPS chauffeur : validation des coordonnées, mise à jour de la base de données et rafraîchissement temps réel du dashboard.
- Consultation parent : récupération des notifications et affichage dans l'application Android.

C.3 Diagrammes de Classes

Le diagramme de classes du domaine représente les principales entités métier : rôles utilisateurs, bus, trajet, élève et notification parent. Il décrit aussi les relations entre elles, comme un bus transportant des élèves, un trajet affecté aux élèves et un élève recevant des notifications.

Le diagramme de structure backend montre l'organisation technique du backend en routes, contrôleurs, services, modèles, middlewares, accès base de données et service temps réel.

C.4 Recommandation d'Integration

Pour la version PDF finale, les diagrammes Mermaid de `project-archive/UML_DIAGRAMS.md` peuvent être exportés en PNG ou PDF, placés dans le dossier `figures/`, puis inclus avec `\includegraphics`. Cette méthode garde la mise en page du rapport stable tout en conservant les sources modifiables des diagrammes.



المدرسة العليا
للأساتذة - مراكش
ÉCOLE NORMALE
SUPÉRIEURE - MARRAKECH



Rhendour Zakaria

Résumé du Projet

SafeBus est une plateforme web et mobile de suivi du transport scolaire développée autour d'une idée simple : les parents, les chauffeurs et l'administration ne devraient pas dépendre uniquement des appels téléphoniques pour connaître l'état d'un bus scolaire. Le projet combine un backend Express, un tableau de bord React, une application Android native, une base de données MySQL et des mises à jour en temps réel avec Socket.IO. Il permet de gérer les élèves, les bus, les trajets, les actions du chauffeur, les positions GPS et les notifications aux parents dans un environnement local basé sur Docker.

Mots-clés : transport scolaire, tableau de bord web, application Android, notifications en temps réel, développement full-stack